Ⅰ. 事不过三

时间限制： 2.0 秒

本题为**通信题**。

请务必在每次输出后刷新标准输出缓冲区。各编程语言的刷新方式如下：

- 对于 C/C++，请使用 `fflush(stdout)` 或 `cout.flush()`；
- 对于 Java，请使用 `System.out.flush()`；
- 对于 Python，请使用 `sys.stdout.flush()`。

【题目描述】

小 L 与小 S 是一对关系非常要好的朋友。

有一天，小 L 看到了一个长度为 n 的，只包含字符 `0` 与 `1` 的字符串 P，她需要将 P 的内容传递给小 S。

小 L 有一台打字机，这台打字机可以向小 S 发送一个只包含字符 `0` 与 `1` 的字符串 Q，但由于打字机年久失修，传递的字符串 Q 有着以下两个限制：

1. Q 的长度不能大于 $\lfloor \frac{3}{2}n \rfloor$，其中 $\lfloor x \rfloor$ 代表不大于 x 的最大整数。
2. Q 中任何连续的三个字符不能全部相同，即不存在正整数 $1 \le i \le m-2$，使得 $Q_i = Q_{i+1} = Q_{i+2}$，其中 m 为 Q 的长度。字符串的下标从 1 开始。

请帮助小 L 与小 S 制定传递策略，使得小 S 可以根据小 L 发送的字符串 Q 推断出小 L 看到的字符串 P。

【实现细节】

你的程序将会**独立运行两次**，其中第一次将扮演小 L，需要根据看到的字符串 P 向小 S 传递一个符合限制的字符串 Q；第二次将扮演小 S，需要根据收到的字符串 Q 推断出小 L 看到的字符串 P。交互器将在你的程序之间传递信息。

在第一次运行中，你的程序会首先收到字符串 P 的长度与内容，然后你的程序需要输出字符串 Q 的长度与内容，从而向第二次运行传递信息。

在第二次运行中，你的程序会从交互器收到来自第一次运行的信息，即字符串 Q 的长度与内容。你需要根据这些信息推断出其对应字符串 P 的内容。

【输入输出格式】

每个测试点中包含多组测试数据。输入的第一行包含两个正整数 T, r ($1 \le T \le 2 \times 10^4, r \in \{1, 2\}$)，表示数据组数与阶段编号。对于每组测试数据：

- 若 $r = 1$，则输入格式如下：
  - 第一行输入一个正整数 n ($1 \le n \le 5 \times 10^5$)，表示字符串 P 的长度。
  - 第二行输入一个长度为 n 的，仅包含字符 `0` 与 `1` 的字符串 P。

你需要按如下格式输出两行：

- 第一行包含一个正整数 m $(1 \le m \le \lfloor \frac{3}{2}n \rfloor)$，代表字符串 Q 的长度。
- 第二行包含一个长度为 m 的，仅包含字符 `0` 与 `1` 的字符串 Q，其中任何连续的三个字符不能全部相同。

对于每一组测试数据，你必须输出字符串 Q 并刷新标准输出缓冲区后，才能继续输入下一组测试数据。

保证单个测试点内所有测试数据 n 的和不超过 5×10^5。

- 若 $r = 2$，则输入格式如下：
  - 第一行输入一个正整数 m，表示字符串 Q 的长度。
  - 第二行输入一个长度为 m 的，仅包含字符 `0` 与 `1` 的字符串 Q。

  你需要按如下格式输出两行：
  - 第一行包含一个正整数 n，代表字符串 P 的长度。
  - 第二行包含一个长度为 n 的，仅包含字符 `0` 与 `1` 的字符串 P，代表小 L 看到的字符串。

  对于每一组测试数据，你必须输出字符串 P 并刷新标准输出缓冲区后，才能继续输入下一组测试数据，并且单个测试点内测试数据输入的先后顺序相较第一次运行时**可能不同**。

【样例】

第一次运行时，输入如下：

```
3 1
1
0
3
111
6
001100
```

小 L 可以按如下输出发送字符串：

```
1
0
4
0010
7
0101010
```

根据上述输出，第二次运行时，一种可能的输入如下：

```
3 2
4
0010
7
0101010
1
0
```

通过心灵感应，小 S 推断出三组测试数据的 P，按如下格式输出：

```
3
111
6
001100
1
0
```

注意，在第一次运行的第二组测试数据中，小 L 不能按如下方式发送字符串 Q：

```
5
10101
```

Q 的长度 $m = 5$ 大于 $\lfloor \frac{3}{2}n \rfloor = 4$，违反限制 1。

```
4
1110
```

Q 中前三个字符 Q_1, Q_2, Q_3 均为 `1`，违反限制 2。

【程序测试方式】

下发的脚本文件 `triplet_testing_tool.py` 可以协助你进行本地测试，并打印详细的交互过程。测试时，将脚本文件与你编译生成的可执行文件置于同一文件夹下，然后在终端内运行以下命令：

```
python3 triplet_testing_tool.py [--quiet] <data_file> <program>
    <arguments>
```

其中：

- `-q, --quiet` 为可选参数，使用该参数后，测试工具不会打印详细的交互过程。
- `data_file` 为你提供给测试脚本的输入数据文件路径。该文件需满足以下格式：

  - 第一行包含两个非负整数 $T, seed$，分别代表测试数据组数，与用于打乱测试数据的随机数种子。若指定 $seed = 0$，则代表不进行打乱。
  - 每组数据的格式与前文描述的“第一次运行”的输入格式完全相同。

- `program` 为你编译生成的可执行文件路径。
- `arguments` 为传递给该可执行文件的额外命令行参数。

关于测试脚本的更多实现细节，请直接查阅其源代码。

注意：

1. 该测试工具与实际评测时的交互库实现**不完全相同**，本地测试结果不具有终断性，仅作调试阶段参考。
2. 测试工具**不会监控**程序的时间与空间消耗，无法用于测试是否符合题目的时空限制。